7주차 Http Study

Cache(2)

저작자: 진승환

지난 주 복습: 원 서버가 캐시에 보내는 두 헤더

원 서버 → 응답 헤더 `Cache-Control:` · `Expires:` → **캐시**

캐시가 사본을 얼마나 오래 써도 되는지 알려준다

`Expires: 0`

항상 만료된 상태

- Expires는 만료 시각을 절대 날짜로 지정
- 0처럼 잘못된 날짜는 이미 지난 시각으로 처리
- 캐시는 사본을 쓰기 전에 매번 원 서버에 재검사

`Cache-Control: max-age=N`

N초 동안 신선

- 응답이 만들어진 뒤 N초 동안 원 서버에 묻지 않고 캐시 사본 사용
- 상대 시간이라 서버·캐시 시계 차이에 영향 없음
- Expires와 함께 오면 max-age가 우선

`Cache-Control: must-revalidate`

만료되면 반드시 재검사

- 신선도가 지난 사본은 원 서버 확인 없이 제공 금지
- 원 서버에 연결할 수 없으면 오래된 사본 대신 504 오류 반환
- 사본이 신선한 동안에는 재검사 없이 그대로 사용

LM 인자 알고리즘: 오래 안 바뀐 문서는 오래 캐시

응답에 `Cache-Control`도 `Expires`도 없을 때, 캐시가 신선도를 스스로 추정하는 휴리스틱
기준은 문서의 마지막 변경 시각(`Last-Modified`)

마지막 변경이 오래전

안정적인 문서 → 오래 캐시

- 갑자기 바뀔 가능성이 낮은 문서
- 캐시에 더 오래 보관해도 안전
- 재검사 주기를 길게

마지막 변경이 최근

자주 바뀌는 문서 → 짧게 캐시

- 앞으로도 자주 변경될 가능성이 높은 문서
- 짧은 기간만 캐시
- 곧 서버에 재검사

변경 후 오래될수록 캐시 기간도 길게 → **이 원리를 계산식으로 만든 것이 LM 인자 알고리즘**

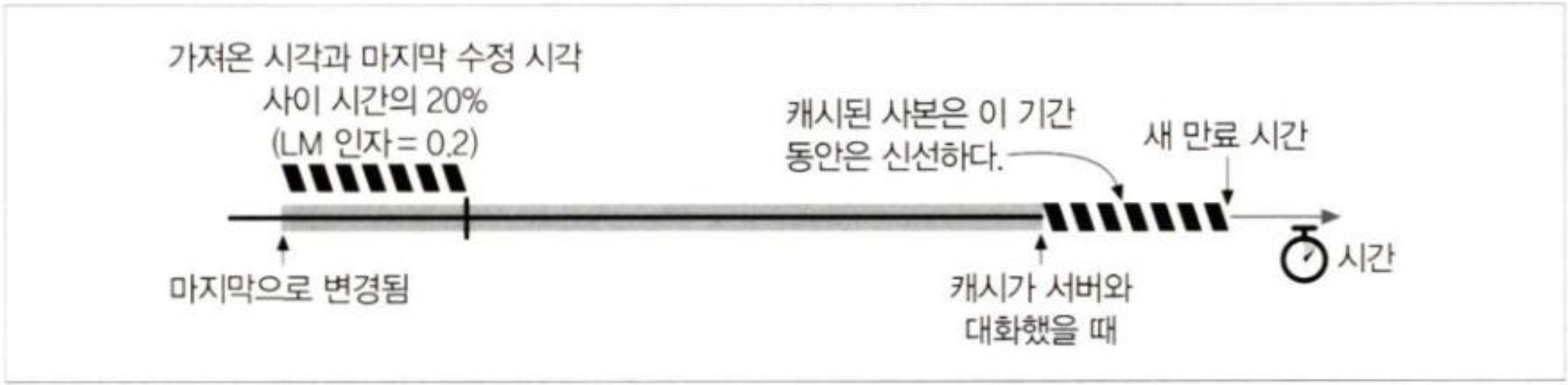


그림 7-16 LM 인자 알고리즘을 이용한 신선도 유지기간 계산

클라이언트가 캐시에게 보내는 Cache-Control 지시자

응답 지시자는 서버가 **"이렇게 다뤄라"**, 요청 지시자는 클라이언트가 **"이런 사본을 원한다"**

덜 엄격
속도·가용성 우선

더 엄격
정확성·보안 우선

지시자	예시	의미	한마디로
`max-stale`	`Cache-Control: max-stale=300`	유효 기간이 지난 뒤 300초까지는 낡은 사본도 받겠다 (값 생략 시 얼마나 낡아도 OK)	**"좀 낡아도 괜찮아"**
`min-fresh`	`Cache-Control: min-fresh=60`	지금부터 최소 60초는 더 신선할 사본만 달라	**"곧 만료될 건 싫어"**
`no-cache` 주의	`Cache-Control: no-cache`	사본을 쓰기 전에 반드시 원 서버에 재검사, 받은 응답으로 캐시 사본도 갱신	**"확인하고 줘"**
`no-store` 주의	`Cache-Control: no-store`	이 요청과 응답을 저장하지 말 것 (기존 사본은 그대로)	**"기록 남기지 마"**
`only-if-cached`	`Cache-Control: only-if-cached`	원 서버에 가지 말고 캐시에 있는 것만 줘, 없으면 504 Gateway Timeout	**"캐시에 있는 것만"**

별도 축
서버 통신 금지

실제로 보이는 곳 브라우저 강력 새로고침(Ctrl+Shift+R) → `Cache-Control: no-cache` + `Pragma: no-cache` 전송

출처: 『HTTP 완벽 가이드』 7장 캐시 제어

캐시가 숨긴 조회수, 광고 회사는 어떻게 셌나

광고는 **노출 수**로 돈을 받는데, 캐시 적중은 서버 로그에 남지 않는다

캐시 적중 → 요청이 원 서버까지 안 옴 → 서버 로그에 조회 기록 없음 → **노출 수 누락 = 광고 수익 감소**

1 캐시 무력화 Cache Busting

방법

- `Cache-Control: no-cache` 헤더 부착
- URL에 매번 랜덤 값 추가

아이디어

- 요청이 항상 원 서버까지 → **서버가 직접 집계**

2 로그 마이그레이션 Log Migration

방법

책: "이상적인 해결책"

- 요청은 캐시에서 처리
- 캐시의 적중 로그를 나중에 서버로 전달

아이디어

- 서버 요청 없이 **캐시 기록으로 집계**

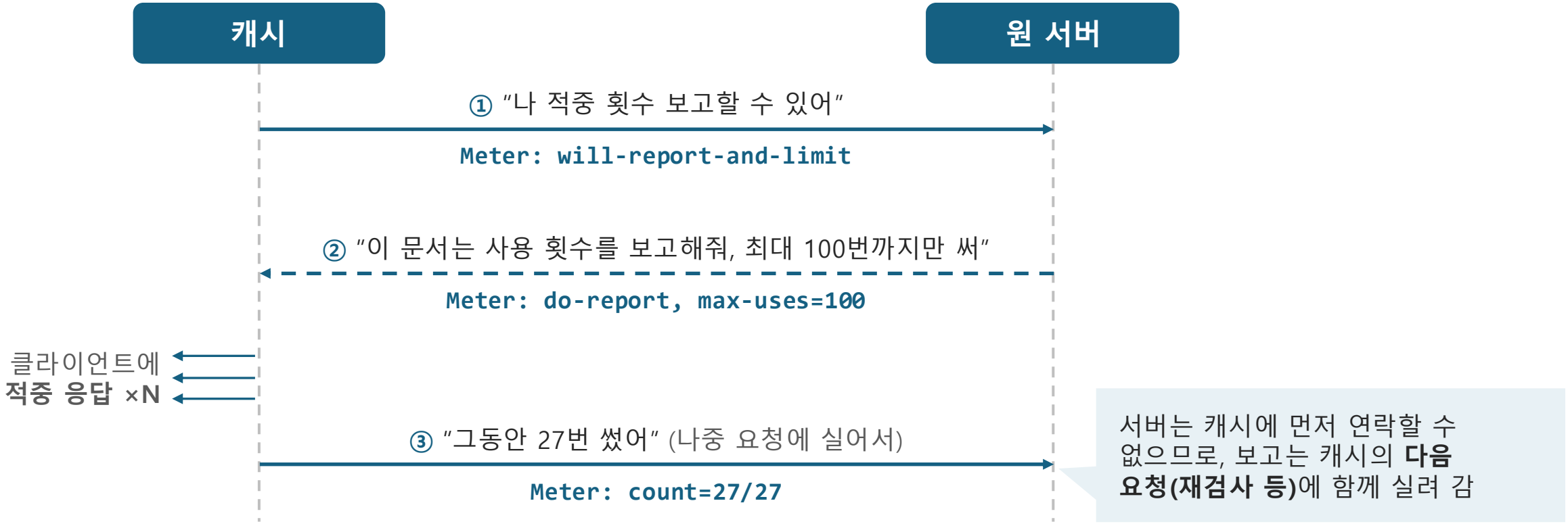


출처: 『HTTP 완벽 가이드』 7장 광고 회사와 캐시

Meter 헤더 — 캐시가 서버에게 적중 횟수를 알려주는 방법

캐시가 조회수를 숨기는 문제를, 서버와 캐시가 **"보고 약속"**으로 풀려는 시도

문제 상황 캐시 적중 → 요청이 원 서버까지 안 옴 → 서버 로그에 조회 기록 없음 → **광고 노출 수 집계 누락**



결과 실험적 제안에 그쳐 거의 구현되지 않음 → 실무는 캐시 무력화(no-cache, 랜덤 URL)나 추적 픽셀로 노출 집계

출처: 『HTTP 완벽 가이드』 7장 광고 회사와 캐시 (RFC 2227)

왜 막혔고, 지금은 어떻게 하나

무거운 광고는 **캐시**하고, 집계는 **가볍게 따로** 한다

두 전략의 한계

1 캐시 무력화의 단점 Cache Busting

- 캐시의 이점(**빠른 응답, 대역폭 절약**) 포기
- 사용자는 **느려지고**, 서버·네트워크 **부담 증가**

2 로그 마이그레이션이 막힌 이유 Log Migration

- 적중 로그가 **너무 커서** 옮기기 어려움
- 제공자별로 나눠 주는 **표준 형식 없음**
- 남의 접속 기록 → **인증·프라이버시** 문제
- 결과: 대형 캐시 업체가 중요 고객에게만 **수작업**으로 전달

오늘날의 방식

참고: 현재 실무

1 추적 픽셀·스크립트 Tracking Pixel

- 광고 본체는 캐시에서, **노출 기록용 작은 요청**(1×1 픽셀, JS 비콘)만 따로 전송
- 캐시 무력화를 **"집계 요청 하나"**로 최소화한 절충

2 CDN 접속 로그·통계 CDN Logs

- 콘텐츠 제공자가 **직접 계약한 캐시** → 로그 제공에 문제 없음
- 로그 마이그레이션 아이디어가 **실제로 자리 잡은 형태**
- 예: CloudFront 액세스 로그

출처: 『HTTP 완벽 가이드』 7장 광고 회사와 캐시. 오른쪽 영역은 책 본문 밖의 현재 실무 사례